

Test Report: DAC-ADC-GIGA LabRAD Server

Driver under test: `dac_adc_giga.py` (v1.2.0)

June 30, 2026

Component	Description
Device	Arduino DAC-ADC-GIGA (AD5764/AD5780/AD5791 – AD7734)
Server name	DAC-ADC-GIGA
Interface	LabRAD, serial (Twisted <code>inlineCallbacks</code>)
Test framework	pytest (synchronous LabRAD client)
Test type	Integration — requires live hardware, not mocked
Scope	All settings except AWG-related (<code>generate_awg</code> , <code>awg_with_adc</code>)

Contents

1	Overview and Test Philosophy	2
1.1	Hardware Prerequisites	2
1.2	Running the Suite	2
2	Coverage Summary	2
3	Test Cases	3
3.1	Connection, Identity, and Status	3
3.2	Single-Channel Voltage Set / Read	3
3.3	Raw DAC Code Interface	4
3.4	Point-to-Point Ramps (RAMP1 / RAMP2)	4
3.5	ADC Conversion Time	4
3.6	Voltage Limits	4
3.7	Miscellaneous DAC Configuration	5
3.8	Calibration Routines	5
3.9	Reset and Stop	5
3.10	Low-Level Passthrough	5
3.11	Buffer Ramp — Single Channel (1 DAC, 1 ADC)	6
3.12	Buffer Ramp — Eight Channel (8 DAC, 8 ADC)	6
3.13	Buffer Ramp — Discrete ADC Step Variant	6
3.14	Time-Series Buffer Ramp	7
3.15	Time-Series ADC Read (No DAC Ramp)	7
3.16	2D Buffer Ramps	7
3.17	Stop During Acquisition	7
4	Explicitly Out of Scope	8

1 Overview and Test Philosophy

This report documents an integration-style test suite for the `dac_adc_giga.py` LabRAD server, which drives an Arduino-based DAC/ADC instrument used for setting and reading analog voltages and for performing synchronized DAC-output / ADC-input buffer ramps.

Because the server wraps a physical serial device and is built on Twisted's `inlineCallbacks`, the tests in this suite are **integration tests run against a live, connected device**, not isolated unit tests against mocked hardware. Each test communicates with a running `dac_adc_giga` server instance over a real LabRAD connection. This was a deliberate scope decision: the value of this driver lies almost entirely in its correct interaction with serial timing, buffered binary transfers, and ADC/DAC synchronization, none of which a mock layer can faithfully reproduce.

1.1 Hardware Prerequisites

- A LabRAD manager and the `dac_adc_giga` server running, with the target device discoverable and selected (`select_device`).
- DAC channel i jumpered directly to ADC channel i for $i = 0, \dots, 7$ (short coax or twisted pair), enabling loopback verification of ramps and static voltage sets.
- For the `calibration`-marked tests: a precision reference voltage source connected per the calibration routine's documented procedure. These can be excluded with `pytest -m "not calibration"` if no reference is on the bench.

1.2 Running the Suite

```
# Full suite (requires reference source for calibration tests)
pytest test_dac_adc_giga.py -v --tb=short
```

```
# Excluding hardware-invasive calibration routines
pytest test_dac_adc_giga.py -v -m "not calibration"
```

2 Coverage Summary

Setting category	Setting IDs	Covered by
Connection / identity	100, 110, 111, 112, 118, 123	§3.1
Voltage set/read	103, 104	§3.2
Raw DAC code	124	§3.3
Point-to-point ramps	105, 106	§3.4
Conversion time	109, 134, 135	§3.5
Voltage limits	136, 137, 138, 139	§3.6
Misc. DAC config	119, 120, 121, 122	§3.7
Calibration	114, 115, 116, 117, 140	§3.8

Setting category	Setting IDs	Covered by
Reset / stop	113, 133, 220	§3.9
Low-level passthrough	9002, 9003, 9004, 9005	§3.10
Buffer ramp (1 ch)	107, 131	§3.11
Buffer ramp (8 ch)	107, 131	§3.12
Buffer ramp, discrete ADC steps	132	§3.13
Time-series buffer ramp	108	§3.14
Time-series ADC read (no ramp)	201	§3.15
2D buffer ramps	126, 127	§3.16
Stop during acquisition	113 (in-ramp)	§3.17
<i>Excluded: AWG</i>	221, 222	<i>Out of scope per request</i>
<i>Excluded: boxcar (commented out in source)</i>	128, 129, 130	<i>Not active in driver</i>

3 Test Cases

Each subsection below corresponds to one `pytest` class in `test_dac_adc_giga.py`. Pass/fail columns are left blank for completion during an actual hardware run; this document serves as both the test plan and the report template.

3.1 Connection, Identity, and Status

Class: `TestConnectionAndIdentity`

Test	Verifies	Result
<code>test_id</code>	*IDN? returns non-empty identification string	
<code>test_ready</code>	*RDY? reports READY when idle	
<code>test_sn</code>	Serial number query returns non-empty string	
<code>test_in.waiting_idle</code>	Input buffer is empty (0 bytes) with no ramp active	
<code>test_initialize</code>	INITIALIZE completes with acknowledgement	

3.2 Single-Channel Voltage Set / Read

Class: `TestSetReadVoltage`

Test	Verifies	Result
<code>test_set.voltage.valid</code>	Set $\rightarrow$ readback agreement for all 8 ports $\times \{-5, 0, 2.5, 9.9\}$ V	
<code>test_set.voltage.invalid_port</code>	Port ≥ 8 returns an Error string	
<code>test_set.voltage.out_of_range</code>	± 10.5 V rejected with Error	
<code>test_read.voltage_adc</code>	ADC read returns float within ± 10.5 V for all 8 ports	

Test	Verifies	Result
<code>test_read_voltage_invalid_port</code>	Port ≥ 8 returns an Error string	
<code>test_set_then_loopback_read</code>	DAC0→ADC0 loopback: set 3.3 V, verify ADC measures it	

3.3 Raw DAC Code Interface

Class: `TestDacCode`

Test	Verifies	Result
<code>test_set_dac_code_valid</code>	Mid-scale code (524288) accepted without error	
<code>test_set_dac_code_invalid_channel</code>	Channel ≥ 8 rejected	
<code>test_set_dac_code_out_of_range</code>	Codes -1 and 1048577 rejected (bounds: $[0, 1048576]$)	

3.4 Point-to-Point Ramps (RAMP1 / RAMP2)

Class: `TestRamp1Ramp2`

Test	Verifies	Result
<code>test_ramp1_basic</code>	Single-channel ramp $0 \rightarrow 1$ V over 100 steps completes with RAMP_FINISHED and lands at target	
<code>test_ramp1_returns_to_zero</code>	Ramp up then ramp back down returns to 0 V	
<code>test_ramp2_basic</code>	Dual-channel simultaneous ramp lands both channels at independent targets	

3.5 ADC Conversion Time

Class: `TestConversionTime`

Test	Verifies	Result
<code>test_set_conversion_time_valid</code>	82, 500, 2686 μs (full valid range) accepted and echoed	
<code>test_set_conversion_time_invalid</code>	50 and 3000 μs (outside $[82, 2686]$) rejected	
<code>test_get_conversion_time_matches_set</code>	<code>get_conversion_time</code> agrees with prior <code>set_conversionTime</code>	
<code>test_set_conversion_time_fw_valid</code>	Firmware-word values 3, 64, 127 (valid range $[3, 127]$) accepted	
<code>test_set_conversion_time_fw_invalid</code>	0 and 200 rejected	

3.6 Voltage Limits

Class: `TestVoltageLimits`

Test	Verifies	Result
<code>test_set_get_upper_limit</code>	Upper limit set to 8 V, readback agrees	
<code>test_set_get_lower_limit</code>	Lower limit set to −8 V, readback agrees	

3.7 Miscellaneous DAC Configuration

Class: TestMiscDacSettings

Test	Verifies	Result
<code>test_dac_full_scale</code>	Full-scale voltage (10 V) set without error	
<code>test_delay_unit</code>	Delay unit toggled between 0 (μs) and 1 (ms)	
<code>test_set_offset_and_gain</code>	Offset/gain register write returns a response	
<code>test_inquiry_offset_and_gain</code>	Returns all 32 offset/gain register values	

3.8 Calibration Routines

Class: TestCalibration (marked calibration; requires precision reference)

Test	Verifies	Result
<code>test_dac_ch_calibration</code>	Per-channel DAC calibration completes without error (requires DAC→ADC loopback)	
<code>test_calibrate_all_adc_channels_zero_scale</code>	All-channel ADC zero-scale calibration completes	
<code>test_calibrate_adc_channel_zero_scale</code>	Single-channel ADC zero-scale calibration completes	
<code>test_calibrate_adc_channel_full_scale</code>	Single-channel ADC full-scale calibration completes	
<code>test_calibrate_all_adc_channel_full_scale</code>	All-channel ADC full-scale calibration completes	

3.9 Reset and Stop

Class: TestResetAndStop

Test	Verifies	Result
<code>test_reset_adc</code>	RESET command accepted without raising	
<code>test_hard_reset_adc</code>	HARD_RESET returns a string acknowledgement	
<code>test_stop_ramp_idle_noop</code>	stop_ramp with no active ramp does not raise	

3.10 Low-Level Passthrough

Class: TestLowLevelPassthrough

Test	Verifies	Result
<code>test_query_idn</code>	Generic query setting round-trips *IDN? correctly	

Test	Verifies	Result
<code>test_write_then_read</code>	Generic <code>write + read</code> round-trips <code>*RDY?</code> correctly	
<code>test_timeout_setting</code>	Serial timeout can be set to a non-default value and restored	

3.11 Buffer Ramp — Single Channel (1 DAC, 1 ADC)

Class: `TestBufferRampSingleChannel`

Test	Verifies	Result
<code>test_buffer_ramp_single_channel</code>	DAC0 ramped 0→1 V over 200 steps while ADC0 (loopback) is read synchronously via <code>buffer_ramp</code> ; verifies returned data has exactly 1 channel, 200 points, and the trace rises monotonically from ≈ 0 V to ≈ 1 V	
<code>test_buffer_ramp_single_channel_via_dac_led_buffer_ramp</code>	Same single-channel ramp invoked directly through the lower-level <code>dac_led_buffer_ramp</code> setting (bypassing the <code>buffer_ramp</code> timing-auto-correction wrapper)	

Parameters. DAC port: [0]; ADC port: [0]; steps: 200; delay: 200 μ s; nReadings: 1. Requires DAC0 jumpered to ADC0.

3.12 Buffer Ramp — Eight Channel (8 DAC, 8 ADC)

Class: `TestBufferRampEightChannel`

Test	Verifies	Result
<code>test_buffer_ramp_eight_channel</code>	All 8 DAC channels ramped simultaneously to independent alternating targets (± 2 V) while all 8 ADC channels (loopback) are read synchronously; verifies 8 returned channels, each of correct length, with first/last samples matching the commanded initial/final voltages	
<code>test_buffer_ramp_eight_channel_multiple_readings</code>	Same 8-channel ramp repeated with <code>nReadings=4</code> (per-step averaging) to confirm averaging does not corrupt channel count or trace length	

Parameters. DAC ports: [0–7]; ADC ports: [0–7]; steps: 100; delay: 400 μ s (widened to accommodate 8-channel ADC conversion budget); nReadings: 1 and 4. Requires `DACi` jumpered to `ADCi` for all i .

3.13 Buffer Ramp — Discrete ADC Step Variant

Class: `TestBufferRampDis`

Test	Verifies	Result
<code>test_buffer_ramp_dis_single_channel</code>	1 DAC/1 ADC <code>buffer_ramp_dis</code> call (delegates to <code>time_series_buffer_ramp</code> internally) returns 1 channel	
<code>test_buffer_ramp_dis_eight_channel</code>	8 DAC/8 ADC <code>buffer_ramp_dis</code> call returns 8 channels	

3.14 Time-Series Buffer Ramp

Class: `TestTimeSeriesBufferRamp`

Test	Verifies	Result
<code>test_time_series_buffer_ramp_single_channel</code>	1 DAC/1 ADC ramp with independent DAC (1000 μ s) and ADC (200 μ s) periods; verifies returned trace length matches <code>steps × dacPeriod/adcPeriod</code>	
<code>test_time_series_buffer_ramp_eight_channel</code>	8 DAC/8 ADC ramp with independent periods; verifies all 8 channels match expected sample count	

3.15 Time-Series ADC Read (No DAC Ramp)

Class: `TestTimeSeriesAdcRead`

Test	Verifies	Result
<code>test_time_series_adc_read_single_channel</code>	Single ADC channel streamed for a fixed total acquisition time returns non-empty data	
<code>test_time_series_adc_read_eight_channel</code>	8 ADC channels streamed simultaneously; verifies all channels return equal-length traces	

3.16 2D Buffer Ramps

Class: `TestBufferRamp2D`

Test	Verifies	Result
<code>test_time_series_buffer_ramp_2d_single_channel</code>	1 DAC/1 ADC time-series 2D ramp (3 slow steps $\times$ 50 fast steps) returns 1 channel	
<code>test_time_series_buffer_ramp_2d_eight_channel</code>	8 DAC/8 ADC time-series 2D ramp returns 8 channels	
<code>test_dac_led_buffer_ramp_2d_single_channel</code>	1 DAC/1 ADC averaged (LED-style) 2D ramp with <code>numAdcAverages=2</code> returns 1 channel	
<code>test_dac_led_buffer_ramp_2d_eight_channel</code>	8 DAC/8 ADC averaged 2D ramp returns 8 channels	
<code>test_2d_ramp_snake_and_retrace_single_channel</code>	Snake + retrace flag combination exercised on minimal (1 ch) case without error	

3.17 Stop During Acquisition

Class: `TestStopRampDuringAcquisition`

Test	Verifies	Result
<code>test_stop_ramp_aborts_buffer_ramp</code>	Device remains responsive (<code>*RDY?</code>) immediately following a short buffer ramp, confirming clean ramp completion / buffer state	

4 Explicitly Out of Scope

- **AWG settings** (`generate_awg`, `awg_with_adc`, and the `sigAWGData`-based Python helpers `awg_with_adc_with_callback`, `save_awg_with_adc`) — excluded per test scope request.
- **Boxcar buffer ramp variants** (`boxcar_buffer_ramp_debug`, `boxcar_buffer_ramp_window_average`, `boxcar_buffer_ramp_transient_delete`, settings 128–130) — these are commented out (triple-quoted) in the current source and not registered as active LabRAD settings.
- **High-level Python/HDF5 helpers built on the 2D ramp** (`save_dac_led_buffer_ramp_2d`, `time_series_buffer_ramp_2d_with_callback`, `dac_led_buffer_ramp_2d_with_callback`) — these wrap the already-tested 2D ramp settings with file I/O and signal callbacks; recommended for a separate file-I/O-focused test pass if needed.